Приднестровский государственный университет им. Т.Г. Шевченко
Физико-технический институт

ПРАКТИЧЕСКИЙ КУРС ВЕБ РАЗРАБОТКИ REACT TYPESCRIPT NODE И SUPABASE

Лабораторный практикум

Разработал:
ст. преподаватель
кафедры ИТ
Бричаг Д.В.

г. Тирасполь
2026 г.

Лабораторная работа 6

Авторизация и сохранение личных записей

Продолжите проект пятой лабораторной. Добавьте регистрацию и вход по email и паролю, защитите личные страницы и перенесите основную сущность из памяти React в Supabase. После перезагрузки и повторного входа пользователь увидит свои записи, а данные другого аккаунта останутся недоступны.

Цель работы

Связать состояние авторизации с маршрутизацией и постоянными данными приложения. Научиться восстанавливать сессию, реагировать на её изменения, освобождать подписку и защищать строки базы через владельца и отдельные политики RLS для чтения, создания, изменения и удаления.

Обязательный результат

Пользователь регистрируется, при необходимости подтверждает email, входит и выходит. Личные страницы недоступны без сессии. Создание, чтение, редактирование и удаление основной сущности работают с Supabase и сохраняются после перезагрузки. Два аккаунта получают разные списки; подмена owner_id отклоняется базой.

Публичный справочник ЛР 5 продолжает загружаться из Supabase. Демонстрационные записи ЛР 2 больше не смешиваются с настоящими и не импортируются автоматически. Связанные сценарии и два параметра поиска появятся в ЛР 7; в этой работе нужно сохранить уже работающий CRUD и первый фильтр.

Теоретическая справка

Пользователь, сессия и состояние интерфейса

Пользователь Supabase Auth имеет стабильный id. Сессия подтверждает вход и содержит короткоживущий access token и refresh token. Клиент supabase-js по умолчанию сохраняет браузерную сессию и обновляет токен. Пароли и токены

не нужно хранить в собственном состоянии, localStorage или таблицах приложения.

Интерфейс различает три состояния: сессия ещё проверяется, пользователь не вошёл, пользователь вошёл. Пока проверка не завершена, нельзя на мгновение показывать личную страницу. onAuthStateChange сообщает об initial session, входе, выходе и обновлении токена; подписку нужно удалить при размонтировании.

Защищённый маршрут улучшает пользовательский сценарий, но не является границей безопасности. Любой человек может отправить запрос без интерфейса. Настоящую защиту выполняет база: роль authenticated получает операции, а RLS добавляет условие owner_id = auth.uid() к каждой из них.

Политики владельца

Для SELECT, UPDATE и DELETE выражение USING определяет доступные существующие строки. Для INSERT выражение WITH CHECK проверяет новую строку. В UPDATE нужны оба условия: USING защищает исходную строку, WITH CHECK запрещает передать её другому владельцу. Для UPDATE также нужна политика SELECT.

Отсутствие строки в результате не всегда означает её отсутствие в базе: RLS может скрыть чужую строку. Поэтому приложение не должно раскрывать, существует ли чужой id. Проверка двумя аккаунтами подтверждает безопасность лучше, чем скрытая кнопка.

Практическая часть

Шаг 1 Подготовьте проект и настройки Auth

Сохраните завершённую ЛР 5. Из frontend выполните npm ci и npm run build. Проверьте справочник Supabase и отсутствие .env.local в Git. Зафиксируйте рабочее состояние до изменения таблицы и маршрутов.

В Dashboard откройте настройки Authentication. Проверьте, разрешён ли вход по email и паролю, включено ли подтверждение email и какой Site URL задан для локальной разработки. Добавьте фактический адрес Vite в разрешённые redirect URLs, например http://localhost:5173. Используйте порт своего процесса.

Не отключайте подтверждение email без решения преподавателя. Если оно включено, после регистрации сессия может отсутствовать до перехода по письму. Форма должна показать понятную инструкцию, а не считать регистрацию завершённым входом. Для проверки двух аккаунтов используйте доступные вам адреса или выданные преподавателем тестовые учётные записи.

Сохраните фактическую настройку подтверждения и локальный URL в docs/progress.md. Пароли проверочных аккаунтов в Git и Moodle не записывайте.

Шаг 2 Откройте личную таблицу только её владельцам

Создайте новую миграцию, например supabase/migrations/202609220001_tasks_owner_policies.sql. Не изменяйте уже применённую миграцию ЛР 5. Для Task Tracker добавьте значение owner_id по умолчанию, выдайте операции только authenticated и создайте четыре отдельные политики:

```
alter table public.tasks
  alter column owner_id set default auth.uid();

revoke all on table public.tasks from anon, authenticated;
grant select, insert, update, delete
  on table public.tasks to authenticated;

create policy "tasks select own"
on public.tasks for select to authenticated
using ((select auth.uid()) is not null
  and (select auth.uid()) = owner_id);

create policy "tasks insert own"
on public.tasks for insert to authenticated
with check ((select auth.uid()) is not null
  and (select auth.uid()) = owner_id);
```

В той же миграции продолжите правилами UPDATE и DELETE. USING и WITH CHECK в обновлении намеренно совпадают: пользователь может менять только свою строку и не может присвоить ей чужой owner_id.

```
create policy "tasks update own"
on public.tasks for update to authenticated
using ((select auth.uid()) is not null
  and (select auth.uid()) = owner_id)
with check ((select auth.uid()) is not null
  and (select auth.uid()) = owner_id);

create policy "tasks delete own"
on public.tasks for delete to authenticated
using ((select auth.uid()) is not null
  and (select auth.uid()) = owner_id);
```

Замените tasks именем основной таблицы. Справочник сохраняет публичный SELECT из ЛР 5. Связанные таблицы будущей ЛР 7 пока оставьте закрытыми, если они ещё не используются. Не создавайте одну policy FOR ALL: отдельные правила проще проверить и развивать.

Для планировщика мероприятий общий каталог events может читаться всеми, но создавать, менять и удалять событие вправе только его владелец. Регистрации остаются личными. Зафиксируйте это исключение отдельными политиками; не применяйте публичный SELECT ко всем личным проектам.

Шаг 3 Примените миграцию и проверьте права

Выполните новую миграцию через SQL Editor в том же проекте. Убедитесь, что RLS по-прежнему включён, anon не получил прав на основную таблицу, а authenticated имеет четыре операции. Старый справочник должен продолжить чтение без входа.

До создания пользователей основная таблица остаётся пустой. Не добавляйте строки через SQL с вымышленным owner_id и не создавайте пользователей вручную в auth.users. Учётные записи появятся через Auth API, а запись получит owner_id из auth.uid().

В README или progress.md опишите простую модель доступа: tags читают все; tasks читает и изменяет только владелец. Для своей темы замените названия. Эта модель должна совпадать с SQL, а не только с описанием.

Шаг 4 Создайте состояние авторизации

Создайте frontend/src/hooks/useAuth.ts. Хук возвращает одно из трёх состояний и использует общий клиент Supabase из ЛР 5. Callback подписки оставьте синхронным: он только переносит полученную сессию в React.

```
import { useEffect, useState } from 'react'
import type { Session } from '@supabase/supabase-js'
import { supabase } from '../lib/supabase'

export type AuthState =
  | { status: 'checking' }
  | { status: 'guest' }
  | { status: 'authenticated'; session: Session }

function fromSession(session: Session | null): AuthState {
  return session
    ? { status: 'authenticated', session }
    : { status: 'guest' }
}
```

Следующий фрагмент продолжает тот же файл. Подписка обрабатывает события, а getSession восстанавливает браузерную сессию при открытии. Если событие успело изменить состояние, поздний начальный ответ его не перезаписывает.

```
export function useAuth(): AuthState {
  const [auth, setAuth] = useState<AuthState>({ status: 'checking' })

  useEffect(() => {
    let active = true
    const { data: { subscription } } = supabase.auth
      .onAuthStateChange((_event, session) => {
        if (active) setAuth(fromSession(session))
      })

    void supabase.auth.getSession().then(({ data, error }) => {
      if (!active) return
      setAuth((current) => current.status === 'checking'
        ? (error ? { status: 'guest' } : fromSession(data.session))
        : current)
    })

    return () => {
      active = false
      subscription.unsubscribe()
    }
  }, [])

  return auth
}
```

getSession подходит здесь для состояния браузерного интерфейса. Решения о доступе к строкам всё равно принимает RLS на основании проверенного токена. Не передавайте Session через URL и не сохраняйте её вручную.

Шаг 5 Сделайте страницу регистрации и входа

Создайте AuthPage с двумя режимами: «Вход» и «Регистрация». Форма хранит email, пароль, pending, error и message. Поле email имеет type="email" и autoComplete="email"; пароль — type="password" и autoComplete="current-password" или "new-password". Кнопки блокируются на время запроса.

В обработчике регистрации проверьте пароль по правилам учебного проекта, затем вызовите signUp. Передайте текущий origin как адрес возврата; он должен находиться в разрешённых URL Dashboard:

```
const { data, error } = await supabase.auth.signUp({
  email: email.trim(),
  password,
  options: { emailRedirectTo: window.location.origin },
})

if (error) throw error
if (data.session) {
  navigate('/tasks', { replace: true })
} else {
  setMessage('Проверьте почту и подтвердите регистрацию.')
}
```

Не определяйте существование аккаунта по тексту ответа: сервис может скрывать эту информацию. После регистрации не предполагайте, что session уже есть. Пользователь с подтверждением email возвращается по разрешённому адресу и при необходимости выполняет вход.

Для режима входа используйте signInWithPassword. После успеха подписка обновит auth, затем можно перейти к списку. Ошибку покажите рядом с формой и не очищайте email.

```
const { error } = await supabase.auth.signInWithPassword({
  email: email.trim(),
  password,
})
if (error) throw error
navigate('/tasks', { replace: true })
```

Не записывайте пароль в console.log, аналитику или собственную таблицу. Не показывайте токены на странице. Для учебной формы достаточно сообщений Supabase, но не раскрывайте дополнительные сведения о наличии чужого аккаунта.

Шаг 6 Защитите маршруты и добавьте выход

Создайте RequireAuth.tsx с Outlet. Пока сессия проверяется, покажите статус. Гостя перенаправьте на /auth; вошедшему пользователю разрешите вложенный маршрут. Все хуки вызываются до условных возвратов.

```
import { Navigate, Outlet, useLocation } from 'react-router-dom'
import type { AuthState } from '../hooks/useAuth'

export function RequireAuth({ auth }: { auth: AuthState }) {
  const location = useLocation()
  if (auth.status === 'checking') {
    return <p role="status">Проверяем сессию…</p>
  }
  if (auth.status === 'guest') {
    return <Navigate to="/auth" replace
      state={{ from: location.pathname }} />
  }
  return <Outlet />
}
```

В App вызовите useAuth один раз. Маршрут /auth оставьте публичным, а список, создание, карточку и редактирование вложите в Route с RequireAuth. Не удаляйте обработку неизвестного адреса. Если вошедший пользователь открывает /auth, можно перенаправить его к списку.

В общем layout покажите email текущего пользователя и кнопку выхода. Для выхода из текущего браузерного сеанса используйте явный local scope. Подписка получит SIGNED_OUT и очистит auth:

```
async function handleSignOut() {
  const { error } = await supabase.auth.signOut({ scope: 'local' })
  if (error) throw error
}
```

После выхода личный список должен сразу исчезнуть из состояния, а прямое открытие /tasks/:id — перенаправить на вход. Не очищайте всё localStorage вручную: клиент Supabase управляет своими данными сессии.

Шаг 7 Опишите строки БД и преобразование данных

Создайте frontend/src/services/tasks.ts. Тип строки повторяет имена столбцов PostgreSQL. В интерфейсе сохраняются знакомые camelCase и поле tag. Преобразование находится в сервисе, поэтому компоненты не зависят от snake_case базы.

```
type TaskRow = {
  id: string
  owner_id: string
  tag_id: string
  title: string
  description: string
  status: Task['status']
  due_date: string
  priority: Task['priority']
  created_at: string
}

const taskColumns = 'id, owner_id, tag_id, title, description, status, due_date,
priority, created_at'

function fromRow(row: TaskRow): Task {
  return { id: row.id, title: row.title,
    description: row.description, status: row.status,
    dueDate: row.due_date, priority: row.priority, tag: row.tag_id }
}
```

Для записи подготовьте объект без id, owner_id и created_at. owner_id задаёт база через auth.uid(); клиент не должен выбирать владельца в обычной форме.

```
function toWrite(draft: TaskDraft) {
  return {
    tag_id: draft.tag,
    title: draft.title.trim(),
    description: draft.description.trim(),
    status: draft.status,
    due_date: draft.dueDate,
    priority: draft.priority,
  }
}
```

В примерах ниже overrideTypes уточняет статический тип ответа, но не проверяет данные во время выполнения. Структура защищена нашей схемой, а значения — ограничениями БД. Если типы проекта генерируются через Supabase CLI, используйте сгенерированный Database вместо ручного TaskRow.

Шаг 8 Реализуйте чтение и создание

В services/tasks.ts добавьте чтение списка. Фильтр owner_id в запросе не обязателен для безопасности: RLS уже добавляет условие владельца. Можно оставить его для ясности, но нельзя считать заменой политики.

```
export async function getTasks(): Promise<Task[]> {
  const { data, error } = await supabase
    .from('tasks')
    .select(taskColumns)
    .order('created_at', { ascending: false })
    .overrideTypes<TaskRow[], { merge: false }>()

  if (error) throw new Error(`Не удалось загрузить задачи: ${error.message}`)
  return data.map(fromRow)
}
```

Создание возвращает сохранённую строку с id базы. INSERT не содержит owner_id: значение по умолчанию берётся из текущей сессии, а WITH CHECK подтверждает владельца.

```
export async function createTask(draft: TaskDraft): Promise<Task> {
  const { data, error } = await supabase
    .from('tasks')
    .insert(toWrite(draft))
    .select(taskColumns)
    .single()
    .overrideTypes<TaskRow, { merge: false }>()

  if (error) throw new Error(`Не удалось создать задачу: ${error.message}`)
  return fromRow(data)
}
```

В App уберите импорт initialTasks и начальное заполнение демонстрационными данными. До входа tasks — пустой массив. После появления session.user.id загрузите getTasks; во время ожидания покажите loading, при ошибке — сообщение и повтор. При смене пользователя очистите старый список до нового запроса.

Обработчик создания становится асинхронным: вызовите createTask, затем повторно загрузите список и верните id созданной строки. NewTaskPage ждёт Promise, после успеха открывает карточку. Форма блокирует повторную отправку и сохраняет черновик при ошибке.

Шаг 9 Подключите изменение и удаление

Добавьте в сервис две операции. UPDATE всегда ограничивайте id. Не отправляйте owner_id вместе с обычными изменяемыми полями.

```
export async function updateTask(
  id: string, draft: TaskDraft,
): Promise<void> {
  const { error } = await supabase
    .from('tasks')
    .update(toWrite(draft))
    .eq('id', id)
  if (error) throw new Error(`Не удалось изменить задачу: ${error.message}`)
}

export async function deleteTask(id: string): Promise<void> {
  const { error } = await supabase.from('tasks').delete().eq('id', id)
  if (error) throw new Error(`Не удалось удалить задачу: ${error.message}`)
}
```

После успешного изменения или удаления повторно загрузите список, затем выполните прежнюю навигацию. При ошибке оставайтесь на странице и покажите сообщение. В форме введённые значения сохраняются, кнопка повторной отправки снова становится доступной.

Измените контракты страниц: onCreate возвращает Promise<string>, onUpdate и onDelete — Promise<void>, TaskForm ожидает асинхронный onSave. Добавьте pending и submitError. Кнопки сохранения, удаления и повторного действия блокируются на время соответствующего запроса.

```
try {
  setPending(true)
  setSubmitError('')
  await props.onSave({ ...draft, title: draft.title.trim() })
} catch (error: unknown) {
  setSubmitError(error instanceof Error
    ? error.message : 'Не удалось сохранить запись.')
} finally {
  setPending(false)
}
```

Предметная валидация ЛР 3 и проверка доступного справочника ЛР 4–5 выполняются до сетевого запроса. Нажатие «Отмена» не отправляет update. Подтверждение удаления остаётся перед вызовом deleteTask.

Шаг 10 Очистите данные при смене сессии

Загрузка списка зависит от session.user.id. Если auth становится guest, сразу установите tasks в [] и сбросьте состояние загрузки. При входе другого пользователя сначала очистите прежний массив, затем получите новый. Так интерфейс не показывает данные предыдущего аккаунта даже на мгновение.

Не сохраняйте личный список в localStorage как резерв и не возвращайте initialTasks при ошибке. Ошибка БД должна быть видна пользователю. Публичный справочник можно загружать до входа, но форма основной сущности находится за RequireAuth.

При обновлении страницы auth сначала имеет status checking. Дождитесь завершения проверки, затем загружайте задачи. Не запускайте getTasks для guest и не отправляйте запросы при каждом рендере.

Шаг 11 Проверьте два аккаунта и реальные границы доступа

Подготовьте аккаунты A и B и завершите подтверждение email согласно настройке проекта. Проверяйте в одном браузере последовательно либо в двух профилях. Не передавайте пароли другому студенту.

1. Войдите как A, создайте две записи, обновите страницу и измените одну. Запишите id одной строки.
2. Выйдите: список очищается, личный URL перенаправляет на /auth. Войдите снова как A и увидьте изменения.
3. Войдите как B: список A не виден. Создайте запись B и обновите страницу.
4. Из интерфейса B временно запросите select с owner_id пользователя A — результат не должен содержать строки A.
5. Временно попробуйте update или delete по сохранённому id A. Чужая строка не меняется; RLS может вернуть ноль затронутых строк без раскрытия её существования.

6. Временно попробуйте insert с явно переданным owner_id пользователя A. WITH CHECK должен отклонить запрос. Удалите проверочный код.
7. Вернитесь к A и убедитесь, что его записи не изменены, а запись B не видна.
8. Проверьте создание, редактирование, отмену, удаление, первый фильтр и справочник.

В Dashboard администратор проекта видит строки обоих пользователей — это ожидаемо и не доказывает доступ браузера. Доказательство состоит из запросов с сессиями A и B. Не используйте secret key или service_role для клиентской проверки: они обходят RLS.

Если UPDATE чужого id не возвращает ошибку, проверьте фактические данные после запроса. Без доступа SELECT чужая строка скрыта и операция может затронуть ноль строк. Интерфейс не должен сообщать пользователю, что чужой id существует.

Шаг 12 Проверьте ошибки, сборку и Git

1. Введите неправильный пароль: форма показывает ошибку и сохраняет email.
2. При включённом подтверждении зарегистрируйте аккаунт и проверьте состояние без session.
3. Откройте личный URL без входа и во время checking: данные не видны.
4. Временно выключите сеть при загрузке списка и сохранении: видна ошибка, черновик не потерян.
5. Быстро нажмите «Сохранить» дважды: повторный запрос заблокирован.
6. Завершите сессию и проверьте очистку списка.
7. Верните сеть, войдите и повторите действие.
8. Выполните npm run build и npm run preview.

Обновите README и docs/progress.md: настройка Auth и redirect URL, состояние подтверждения email, маршруты, модель доступа, порядок применения

новой миграции и сценарий двух аккаунтов. В .env.example остаются только имена URL и publishable key; реальные пароли, токены и .env.local не коммитятся.

```
git status
git add .
git diff --cached --stat
git commit -m "Lab 6: add auth and persistent personal records"
git push
```

В чистой копии выполните npm ci и npm run build из frontend, создайте .env.local по образцу и пройдите вход. Преподавателю передайте репозиторий и коммит ЛР 6. Проверочные пароли не входят в отчёт.

Частые проблемы

После регистрации сразу открывается личная страница без сессии. Проверяйте data.session. Если подтверждение email включено, покажите инструкцию и дождитесь подтверждения или последующего входа.

После обновления на мгновение видны личные страницы гостя. Не считайте отсутствие session гостем до завершения initial checking. RequireAuth должен показывать ожидание, пока useAuth не вернул guest или authenticated.

Пользователь вошёл, но SELECT возвращает permission denied. Проверьте grant для authenticated, отдельную SELECT policy, включённый RLS и owner_id строк. Не ослабляйте условие до using (true).

INSERT нарушает NOT NULL для owner_id. Проверьте default auth.uid(), наличие действующей сессии и вызов через общий браузерный клиент. Не подставляйте случайный UUID.

UPDATE не работает, хотя строка своя. Для UPDATE нужна также SELECT policy. Проверьте USING и WITH CHECK, права update и отсутствие owner_id в изменяемом объекте.

После выхода остаются задачи предыдущего пользователя. Очистите tasks при guest и перед загрузкой нового user.id. Не полагайтесь только на размонтирование страницы.

Создание дублируется. Блокируйте submit на время Promise и не вызывайте onSave из onChange. Функция обновления состояния React не должна выполнять сетевой запрос.

Использование ИИ при выполнении работы

Можно попросить ИИ объяснить событие Auth, ошибку запроса или конкретную политику. Передавайте обезличенный код и текст ошибки. Не передавайте email вместе с паролем, access/refresh token, содержимое .env.local, secret key или service_role. Самостоятельно проверьте доступ двумя настоящими сессиями.

Результаты работы и сдача

В репозитории находятся миграция политик владельца, Auth-форма, состояние сессии, защита маршрутов, постоянный CRUD и инструкции. После входа пользователь видит свои данные и сохраняет изменения в Supabase. После выхода список очищается. Два аккаунта не читают и не изменяют записи друг друга.

На демонстрации зарегистрируйтесь или войдите, создайте запись, обновите страницу, измените и удалите её. Выйдите и откройте личный URL. Затем войдите вторым аккаунтом и покажите отдельный список, запрет подмены owner_id и сохранность данных первого пользователя. Объясните роли RequireAuth и RLS.

Критерии оценки

Максимум — 10 баллов. Скрытие чужих записей только в интерфейсе без RLS не выполняет обязательный результат. Локальный массив и localStorage не заменяют Supabase.

Функционал — 5 баллов: регистрация, вход, подтверждение-aware сценарий и выход (1); восстановление сессии и защищённые маршруты (1); чтение и

создание личных записей в БД (1); редактирование и удаление с повторной загрузкой (1); данные сохраняются после перезагрузки и повторного входа (1).

Ошибки и безопасность — 2 балла: loading, auth/CRUD errors и блокировка повторной отправки без потери формы (1); четыре политики владельца, закрытый anon-доступ и успешная проверка двумя аккаунтами, включая подмену owner_id (1).

Непрерывность проекта — 1 балл: справочник, форма, карточка, отмена, фильтр и маршруты прошлых работ сохранены; демонстрационные записи не смешиваются с личными.

Git и воспроизводимость — 2 балла: миграция, Auth и CRUD-код доступны в коммите (1); README, progress.md и .env.example актуальны, чистая копия собирается и запускается после настройки (1).

Вопросы для защиты

1. Чем пользователь отличается от сессии?
2. Почему после signUp нельзя всегда считать пользователя вошедшим?
3. Зачем нужен отдельный статус checking?
4. Почему RequireAuth не заменяет RLS?
5. Чем USING отличается от WITH CHECK?
6. Почему UPDATE требует также SELECT policy?
7. Как список очищается при выходе и смене аккаунта?
8. Как вы доказали, что пользователь B не может изменить запись A?

Материалы для самостоятельной работы

1. Supabase JavaScript — Auth overview:
   https://supabase.com/docs/reference/javascript/auth
2. Supabase JavaScript — signUp:
   https://supabase.com/docs/reference/javascript/auth-signup

3. Supabase JavaScript — signInWithPassword:
https://supabase.com/docs/reference/javascript/auth-signinwithpassword
4. Supabase JavaScript — getSession:
https://supabase.com/docs/reference/javascript/auth-getsession
5. Supabase JavaScript — onAuthStateChange:
https://supabase.com/docs/reference/javascript/auth-onauthstatechange
6. Supabase JavaScript — signOut:
https://supabase.com/docs/reference/javascript/auth-signout
7. Supabase — Row Level Security:
https://supabase.com/docs/guides/database/postgres/row-level-security
8. Supabase JavaScript — TypeScript support:
https://supabase.com/docs/reference/javascript/typescript-support

Документация проверена при подготовке работы. Названия разделов Dashboard могут меняться; ориентируйтесь на Authentication, URL Configuration и SQL Editor своего проекта.